MODULUS ERP

Software Requirements Specification

Role-Based SaaS ERP Platform

Document Version	1.0 — Final Draft
Status	Ready for Development
Date	April 2026
Classification	CONFIDENTIAL
Stack	Node.js · Express · MongoDB · React
Deployment	Render (Backend) · Vercel (Frontend)

Engineered for scalable, role-scoped enterprise resource planning.

1. Introduction & System Overview

MODULUS ERP is a cloud-native, role-based enterprise resource planning (ERP) platform designed as a SaaS solution. It provides organizations with a centralized system to manage employees, task workflows, and leave management under a strict role-access model. The system enforces business rules at multiple layers — middleware, service layer, and database — ensuring security and data integrity without redundancy.

1.1 Purpose

This document defines all functional requirements, API specifications, database schemas, frontend page structures, and system architecture for MODULUS ERP. It serves as the engineering reference guide for backend and frontend developers, QA engineers, and system architects.

1.2 Scope

- Full backend REST API with RBAC enforcement
- Three system roles: Admin, Manager, Employee
- Modules: Authentication, User Management, Task Management, Leave Management, Dashboard Analytics
- Frontend SPA (React + Tailwind CSS) deployed on Vercel
- Backend (Node.js + Express) deployed on Render with horizontal scaling via Nginx
- Primary database: MongoDB Atlas with optional Redis caching layer

1.3 Technology Stack

Layer	Technology	Deployment
Frontend	React.js + Tailwind CSS + Axios	Vercel
Backend	Node.js + Express.js	Render (multi-instance)
Database	MongoDB Atlas	Cloud (managed cluster)
Caching	Redis (optional)	Cloud
Load Balancer	Nginx	Conceptual scaling layer
Auth	JWT + Bcrypt	Stateless tokens
Validation	express-validator	Middleware layer
Logging	Winston	Middleware layer

1.4 System Roles

The system supports exactly three roles with hierarchical permissions:

Role	Created By	Scope	Primary Responsibilities
Admin	Seeded (manual)	Global — all data	Create managers, view all tasks/leaves, system-wide analytics
Manager	Admin only	Team / Department	Create employees, assign tasks, approve leaves, team analytics
Employee	Manager only	Own data only	View own tasks, apply for leave, view own dashboard

2. Authentication Architecture

MODULUS ERP follows a closed-registration model. There is no public signup. All users are created within the system by privileged roles. Authentication is stateless, token-based, using JSON Web Tokens (JWT).

2.1 User Creation Flow

The hierarchy is strictly top-down:

1. Admin is seeded directly into the database via a script or seed file (manual, one-time setup)
2. Admin authenticates and creates Manager accounts via POST /users/create
3. Manager authenticates and creates Employee accounts via POST /users/create
4. Employees cannot create any users

2.2 Login Flow

Endpoint: POST /auth/login

Request Body:

```
{ "email": "user@company.com", "password": "SecurePass123!" }
```

Server Execution Steps:

5. Validate request body using express-validator
6. Find user document in MongoDB by email
7. Return 401 if user not found
8. Compare submitted password with bcrypt hash stored in DB
9. Return 401 if password mismatch
10. Generate JWT with payload: { id, role, organizationId }
11. Return token + user metadata in response

Success Response (200):

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { "id": "64f2a...",
"name": "Riya Shah", "role": "manager", "email": "riya@corp.com" } }
```

2.3 JWT Payload Structure

```
{ "id": "64f2a...", "role": "manager", "organizationId": "org_xyz", "iat":
1710000000, "exp": 1710086400 }
```

Token expiry: 24 hours (configurable via environment variable JWT_EXPIRES_IN).

2.4 Token Usage by Frontend

After login, the frontend (React) stores the token in memory (AuthContext) and attaches it to every subsequent API request via an Axios interceptor:

```
Authorization: Bearer <token>
```

The frontend also extracts the role from the decoded token to conditionally render protected routes (RoleProtectedRoute component).

3. Middleware Execution Architecture

Every protected request flows through a deterministic middleware chain before reaching the controller. This enforces security at the request level before any business logic executes.

3.1 Middleware Chain (Request Lifecycle)

For every protected route, the chain is:

Step	Middleware/Handler	File	Responsibility
1	authMiddleware	middleware/auth.middleware.js	Extract & verify JWT; attach req.user = { id, role }
2	roleMiddleware	middleware/role.middleware.js	Check if req.user.role is in allowedRoles array
3	express-validator	(inline route definition)	Validate & sanitize request body/params
4	Logging (Winston)	middleware/logger.middleware.js	Log request details, timing, and errors
5	Controller	modules/[module]/controller	Parse request, call service, format response
6	Service Layer	modules/[module]/service	Execute business logic, DB queries, role filtering
7	MongoDB	modules/[module]/model	Persist or retrieve data
8	Response	controller	Return HTTP response with status code + JSON

3.2 authMiddleware Logic

File: middleware/auth.middleware.js

```
const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token provided' });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = { id: decoded.id, role: decoded.role };
    next();
  } catch (err) { return res.status(401).json({ error: 'Invalid token' }); }
};
```

3.3 roleMiddleware Logic

File: middleware/role.middleware.js

```
const allowRoles = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ error: 'Access denied' });
  next();
};
```

Usage in route definition:

```
router.post('/tasks/create', authMiddleware, allowRoles('admin','manager'),
createTask);
```

4. RBAC Permission Matrix

All permissions are enforced via the allowRoles() middleware. The following matrix defines every action and which roles are authorized to perform it:

Action / Resource	HTTP Method	Admin	Manager	Employee
Login	POST /auth/login	✅	✅	✅
View own profile	GET /profile	✅	✅	✅
Create manager user	POST /users/create	✅	❌	❌
Create employee user	POST /users/create	✅	✅	❌
View all employees	GET /users	✅	Team only	❌
Create task	POST /tasks/create	✅	✅	❌
Assign task	PATCH /tasks/:id/assign	✅	✅	❌
Update task status	PATCH /tasks/:id/status	✅	✅	Own only
View tasks	GET /tasks	All tasks	Team tasks	Own tasks
Apply for leave	POST /leaves/apply	❌	❌	✅
Approve/Reject leave	PATCH /leaves/:id/approve	✅	✅	❌
View leaves	GET /leaves	All leaves	Team leaves	Own leaves
Dashboard stats	GET /dashboard/stats	Global stats	Team stats	Personal stats
Delete user	DELETE /users/:id	✅	❌	❌

Note: Same-endpoint, different-result pattern — a single GET /tasks endpoint returns different datasets based on req.user.role, resolved inside the service layer, not the route.

5. API Endpoints — Complete Reference

5.1 Authentication Endpoints

POST /auth/login

URL	POST /auth/login
Auth Required	No
Roles Allowed	All (public endpoint)

Request Body:

```
{ "email": "string (required)", "password": "string (required, min 8 chars)" }
```

Success Response 200:

```
{ "token": "JWT string", "user": { "id": "ObjectId", "name": "string", "role":
"admin|manager|employee", "email": "string", "department": "string" } }
```

Error Responses:

- 400 — Validation error (missing/invalid fields)
- 401 — Invalid credentials
- 500 — Internal server error

5.2 User Management Endpoints

POST /users/create

URL	POST /users/create
Auth Required	Yes (Bearer Token)
Roles Allowed	Admin (create manager), Manager (create employee)

Request Body:

```
{ "name": "string", "email": "string", "password": "string", "role":
"manager|employee", "department": "string", "managerId": "ObjectId (required if
role=employee)" }
```

Success Response 201:

```
{ "message": "User created successfully", "user": { "id": "ObjectId", "name":
"string", "role": "string", "email": "string" } }
```

Error Responses:

- 400 — Validation error or email already exists
- 403 — Manager trying to create manager; Employee trying to create any

GET /users

URL	GET /users?page=1&limit=10&department=engineering
Auth Required	Yes
Roles Allowed	Admin (all), Manager (own team)

Query Parameters:

- page — Pagination page number (default: 1)
- limit — Results per page (default: 10, max: 50)
- department — Filter by department (optional)
- search — Search by name or email (optional)

Success Response 200:

```
{ "total": 45, "page": 1, "limit": 10, "data": [ { "id": "...", "name": "...",
"role": "...", "department": "...", "status": "active" } ] }
```

GET /users/:id

URL	GET /users/:id
Auth Required	Yes
Roles Allowed	Admin (any user), Manager (own team), Employee (own profile only)

DELETE /users/:id

URL	DELETE /users/:id
Auth Required	Yes
Roles Allowed	Admin only

Success Response 200:

```
{ "message": "User deleted successfully" }
```

GET /profile

URL	GET /profile
Auth Required	Yes
Roles Allowed	All roles

Returns the authenticated user's own profile. Uses req.user.id from the JWT.

5.3 Task Management Endpoints

POST /tasks/create

URL	POST /tasks/create
Auth Required	Yes
Roles Allowed	Admin, Manager

Request Body:

```
{ "title": "string (required)", "description": "string", "assignedTo": "ObjectId
(required)", "department": "string", "priority": "low|medium|high|critical",
"dueDate": "ISO 8601 date string" }
```

Success Response 201:

```
{ "message": "Task created", "task": { "id": "ObjectId", "title": "string",
"status": "pending", "priority": "string", "assignedTo": "ObjectId", "assignedBy":
"ObjectId", "dueDate": "date" } }
```

GET /tasks

URL	GET /tasks?status=pending&priority=high&page=1&limit=10
Auth Required	Yes
Roles Allowed	All (filtered by role in service layer)

Role-based filtering (resolved in TaskService):

- Admin → returns ALL tasks across all departments
- Manager → returns tasks where department matches manager's department
- Employee → returns tasks where assignedTo === req.user.id

PATCH /tasks/:id/status

URL	PATCH /tasks/:id/status
Auth Required	Yes
Roles Allowed	Admin, Manager (any task); Employee (own task only)

Request Body:

```
{ "status": "pending|in-progress|completed" }
```

PATCH /tasks/:id/assign

URL	PATCH /tasks/:id/assign
Auth Required	Yes
Roles Allowed	Admin, Manager only

Request Body:

```
{ "assignedTo": "ObjectId" }
```

DELETE /tasks/:id

URL	DELETE /tasks/:id
Auth Required	Yes
Roles Allowed	Admin, Manager only

5.4 Leave Management Endpoints

POST /leaves/apply

URL	POST /leaves/apply
Auth Required	Yes
Roles Allowed	Employee only

Request Body:

```
{ "leaveType": "sick|casual|annual|unpaid", "startDate": "ISO 8601 date",
"endDate": "ISO 8601 date", "reason": "string (required, min 10 chars)" }
```

Success Response 201:

```
{ "message": "Leave applied", "leave": { "id": "ObjectId", "status": "pending",
"employeeId": "ObjectId", "leaveType": "string", "startDate": "date", "endDate":
"date" } }
```

PATCH /leaves/:id/approve

URL	PATCH /leaves/:id/approve
Auth Required	Yes
Roles Allowed	Admin, Manager only

Request Body:

```
{ "status": "approved|rejected", "remarks": "string (optional)" }
```

Success Response 200:

```
{ "message": "Leave approved", "leave": { "id": "ObjectId", "status": "approved",
"approvedBy": "ObjectId" } }
```

GET /leaves

URL	GET /leaves?status=pending&page=1&limit=10

Auth Required	Yes
Roles Allowed	All (role-filtered in LeaveService)

Same endpoint, role-differentiated results — Admin sees all, Manager sees team, Employee sees own.

5.5 Dashboard Endpoints

GET /dashboard/stats

URL	GET /dashboard/stats
Auth Required	Yes
Roles Allowed	All roles (different aggregation per role)

Admin Response 200:

```
{ "totalEmployees": 128, "activeTasks": 43, "pendingLeaves": 12, "completedTasks":
210, "departmentStats": [ { "dept": "Engineering", "employees": 32, "activeTasks":
15 } ] }
```

Manager Response 200:

```
{ "teamSize": 8, "teamActiveTasks": 5, "pendingLeaveApprovals": 2,
"teamCompletedTasks": 34 }
```

Employee Response 200:

```
{ "myActiveTasks": 3, "myCompletedTasks": 12, "leaveBalance": 10,
"pendingLeaveRequest": 1 }
```

6. Database Schema Design

MODULUS ERP uses MongoDB Atlas with indexed collections for performance. All collections use Mongoose ODM with schema validation.

6.1 User Schema

Collection: users

```
const UserSchema = new mongoose.Schema({
  name:           { type: String, required: true, trim: true },
  email:          { type: String, required: true, unique: true, lowercase: true },
  password:       { type: String, required: true, minlength: 8, select: false },
  role:           { type: String, enum: ['admin','manager','employee'], required:
true },
  department:     { type: String, required: true },
  managerId:      { type: mongoose.Schema.Types.ObjectId, ref: 'User', default:
null },
  organizationId: { type: mongoose.Schema.Types.ObjectId, ref: 'Organization' },
  status:         { type: String, enum: ['active','inactive'], default: 'active' },
  createdAt:      { type: Date, default: Date.now }
}, { timestamps: true });
UserSchema.index({ email: 1 }, { unique: true });
UserSchema.index({ managerId: 1 });
```

Field	Type	Required	Notes
name	String	Yes	Trimmed, user display name
email	String	Yes	Unique, lowercase, used for login
password	String	Yes	Bcrypt hashed, select: false (never returned in queries)
role	Enum	Yes	admin | manager | employee
department	String	Yes	Used for team-level filtering
managerId	ObjectId → User	Conditional	Required when role=employee; links to manager
organizationId	ObjectId → Org	Yes	Multi-tenant support
status	Enum	No	active (default) | inactive

6.2 Task Schema

Collection: tasks

```
const TaskSchema = new mongoose.Schema({
  title:       { type: String, required: true, trim: true },
  description: { type: String, default: '' },
  assignedTo:  { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
},
  assignedBy:  { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
},
  department:  { type: String, required: true },
  status:      { type: String, enum: ['pending','in-progress','completed'],
default: 'pending' },
  priority:    { type: String, enum: ['low','medium','high','critical'], default:
'medium' },
  dueDate:     { type: Date, required: true },
  createdAt:   { type: Date, default: Date.now }
}, { timestamps: true });
TaskSchema.index({ assignedTo: 1 });
TaskSchema.index({ status: 1 });
```

Field	Type	Required	Notes
title	String	Yes	Short descriptive task name
description	String	No	Detailed task description
assignedTo	ObjectId → User	Yes	The employee the task is assigned to
assignedBy	ObjectId → User	Yes	The admin/manager who created the task
department	String	Yes	Used for manager-level filtering
status	Enum	No	pending (default) | in-progress | completed
priority	Enum	No	low | medium | high | critical
dueDate	Date	Yes	ISO 8601 format

6.3 Leave Schema

Collection: leaves

```
const LeaveSchema = new mongoose.Schema({
  employeeId:  { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
},
  leaveType:   { type: String, enum: ['sick','casual','annual','unpaid'], required:
true },
  startDate:   { type: Date, required: true },
  endDate:     { type: Date, required: true },
  reason:      { type: String, required: true, minlength: 10 },
  status:      { type: String, enum: ['pending','approved','rejected'], default:
'pending' },
```

```
  approvedBy:  { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null
},
  remarks:     { type: String, default: '' },
  createdAt:   { type: Date, default: Date.now }
}, { timestamps: true });
LeaveSchema.index({ employeeId: 1 });
LeaveSchema.index({ status: 1 });
```

Field	Type	Required	Notes
employeeId	ObjectId → User	Yes	The employee who applied for leave
leaveType	Enum	Yes	sick | casual | annual | unpaid
startDate	Date	Yes	Leave start date
endDate	Date	Yes	Leave end date (must be >= startDate)
reason	String	Yes	Min 10 characters
status	Enum	No	pending (default) | approved | rejected
approvedBy	ObjectId → User	No	Populated when manager/admin acts on request
remarks	String	No	Optional manager comment on decision

7. Backend Folder Structure

The backend follows a modular, domain-driven structure with a separation of concerns across routes, controllers, services, and models:

```
src/
├── config/
│   ├── db.js                  # MongoDB connection (mongoose.connect)
│   └── env.js                 # Environment variable loader
│
├── middleware/
│   ├── auth.middleware.js      # JWT verification → req.user
│   ├── role.middleware.js      # allowRoles() factory function
│   ├── validate.middleware.js  # express-validator result handler
│   └── logger.middleware.js    # Winston request logger
│
├── utils/
│   ├── jwt.utils.js            # sign / verify helpers
│   ├── bcrypt.utils.js         # hash / compare helpers
│   └── response.utils.js       # Standard { success, data, error } formatter
│
├── modules/
│   ├── auth/
│   │   ├── auth.routes.js
│   │   ├── auth.controller.js
│   │   └── auth.service.js
│   │
│   ├── users/
│   │   ├── users.routes.js
│   │   ├── users.controller.js
│   │   ├── users.service.js
│   │   └── user.model.js
│   │
│   ├── tasks/
│   │   ├── tasks.routes.js
│   │   ├── tasks.controller.js
│   │   ├── tasks.service.js
│   │   └── task.model.js
│   │
│   ├── leaves/
│   │   ├── leaves.routes.js
│   │   ├── leaves.controller.js
│   │   ├── leaves.service.js
│   │   └── leave.model.js
│   │
│   └── dashboard/
│       ├── dashboard.routes.js
```

```
│       ├── dashboard.controller.js
│       └── dashboard.service.js
│
├── core/
│   ├── base.controller.js      # Shared controller patterns
│   ├── base.service.js         # Shared service patterns
│   └── base.repository.js      # Generic DB query patterns
│
├── routes/
│   └── index.js                # Aggregates all module routes
│
├── app.js                      # Express app setup, middleware mounting
└── server.js                   # HTTP server start, port binding
```

7.1 Controller vs Service Responsibility

Concern	Controller Handles	Service Handles
Input	Parse req.body, req.params, req.query	Receive clean data objects
Validation	Call express-validator results	Assume data is valid
Business Logic	❌ — never	All business rules & conditions
Database	❌ — never directly	All Mongoose queries
Role Filtering	❌ — never	Filter results by req.user.role
Response	Format HTTP status + JSON body	Return data or throw errors

8. Frontend Pages & Component Architecture

The frontend is a React SPA deployed on Vercel. State management uses AuthContext (JWT + role). Navigation is guarded by ProtectedRoute and RoleProtectedRoute components. Axios interceptors automatically attach JWT to every request.

8.1 Auth Pages

LoginPage — /login

This is the single entry point for all users regardless of role.

Element	Type	Behavior
Email field	Input (type=email)	Controlled input, validated client-side
Password field	Input (type=password)	Controlled input, show/hide toggle
Login button	Button	Submits to POST /auth/login; shows spinner on loading
Error message	Alert	Shown on 401 invalid credentials
Role redirect	useEffect	On success: Admin → /admin, Manager → /manager, Employee → /employee

JWT stored in AuthContext (not localStorage). Role decoded from token and stored in context for conditional rendering.

8.2 Admin Pages

AdminDashboard — /admin

Component/Section	Data Source	API Called	Description
Stats Cards	Dashboard aggregation	GET /dashboard/stats	Total employees, active tasks, pending leaves, department count
Department Chart	MongoDB aggregate	GET /dashboard/stats	Bar/pie chart of employee count per department
Recent Activity Feed	Last 10 actions	GET /dashboard/stats	Latest task updates, leave decisions
Quick Actions	Navigation links	N/A	Buttons to create user, create task, view all leaves

EmployeeManagementPage — /admin/employees

Component/Section	API Called	Description
Employee List Table	GET /users	Paginated, searchable, filterable by department and role
Create User Button + Modal	POST /users/create	Form with name, email, password, role, department fields
Employee Detail View	GET /users/:id	Side panel showing employee profile, tasks, leave history
Delete User Button	DELETE /users/:id	Confirmation dialog before deletion
Search/Filter Bar	GET /users?search=&department=	Live search with debounce

TaskManagementPage — /admin/tasks

Component/Section	API Called	Description
Task Table	GET /tasks	All tasks; filterable by status, priority, assignee, department
Create Task Modal	POST /tasks/create	Fields: title, description, assignedTo (dropdown), priority, dueDate
Status Update	PATCH /tasks/:id/status	Inline status dropdown in table row
Reassign Task	PATCH /tasks/:id/assign	Reassign from task detail view
Delete Task	DELETE /tasks/:id	Confirm + delete

LeaveManagementPage — /admin/leaves

Component/Section	API Called	Description
Leave Requests Table	GET /leaves	All employee leave requests with status badges
Approve Button	PATCH /leaves/:id/approve { status:'approved' }	One-click approval with optional remarks input
Reject Button	PATCH /leaves/:id/approve { status:'rejected' }	Reject with mandatory remarks
Leave Calendar	GET /leaves	Optional calendar view of approved leaves by date

AnalyticsDashboardPage — /admin/analytics

Component/Section	API Called	Description
Task Completion Rate Chart	GET /dashboard/stats	Line chart over time — completed vs total tasks
Leave Trends Chart	GET /dashboard/stats	Bar chart of leave applications per month
Department Productivity	GET /dashboard/stats	Heatmap or radar chart per department
Employee Status Summary	GET /users	Active vs inactive employee count

8.3 Manager Pages

ManagerDashboard — /manager

Section	Data Source	API Called	Description
Team Stats Cards	Team aggregation	GET /dashboard/stats	Team size, active tasks, pending approvals
Pending Leave Approvals Widget	LeaveService	GET /leaves?status=pending	Quick approval/rejection from dashboard
Team Task Summary	TaskService	GET /tasks	In-progress and overdue tasks for the team

Manager Task View — /manager/tasks

Component	API	Description
Team Task List	GET /tasks (team filtered)	All tasks for manager's department
Create Task Button + Form	POST /tasks/create	Assign to any employee in own team
Task Status Update	PATCH /tasks/:id/status	Update any team task's status

Manager Leave View — /manager/leaves

Component	API	Description
Team Leave Requests	GET /leaves (team filtered)	All leave requests from own team
Approve/Reject Actions	PATCH /leaves/:id/approve	Act on pending requests with optional remarks

8.4 Employee Pages

EmployeeDashboard — /employee

Section	API	Description
My Task Stats	GET /dashboard/stats	Active, completed, overdue counts for own tasks
Leave Balance Card	GET /dashboard/stats	Annual/casual/sick balance display
Recent Task List	GET /tasks (own)	Last 5 tasks with status badges
Pending Leave Request Alert	GET /leaves (own)	Shows if a leave request is awaiting approval

Employee Task View — /employee/tasks

Component	API	Description
My Task List	GET /tasks (own)	Tasks assigned to self — filterable by status
Update Task Status	PATCH /tasks/:id/status	Employee can mark task as in-progress or completed

Employee Leave View — /employee/leaves

Component	API	Description
Apply for Leave Form	POST /leaves/apply	Fields: type, start date, end date, reason
My Leave History Table	GET /leaves (own)	All past and pending leave requests with status badges
Leave Status Badge	N/A	Pending (yellow) / Approved (green) / Rejected (red)

9. End-to-End Data Flow: Leave Approval

This section traces the complete journey of a leave application from employee submission through manager approval, as a worked example of the full system integration.

Step	Actor / Layer	Action / Event	Tech Component
1	Employee (Browser)	Clicks 'Apply Leave' on LeaveManagementPage	React Component
2	React (AuthContext)	Axios adds Authorization: Bearer <token> to POST /leaves/apply	Axios Interceptor
3	Nginx	Routes request to Instance N (load balanced)	Nginx Load Balancer
4	Express Router	Routes to /leaves module route file	routes/index.js → leaves.routes.js
5	authMiddleware	Verifies JWT, attaches req.user = { id, role: 'employee' }	middleware/auth.middleware.js
6	roleMiddleware	Checks allowRoles('employee') — passes	middleware/role.middleware.js
7	express-validator	Validates leaveType, startDate, endDate, reason fields	validate.middleware.js
8	LeaveController	Parses request, calls leaveService.applyLeave(data, req.user)	leaves.controller.js
9	LeaveService	Creates Leave document, sets status='pending', saves to MongoDB	leaves.service.js
10	MongoDB Atlas	Persists leave document, triggers index on employeeId	MongoDB Atlas
11	Response	Returns 201 { message, leave } back through chain to browser	leaves.controller.js
12	Manager (Browser)	Sees pending request on ManagerDashboard widget	React Dashboard
13	Manager Action	Clicks Approve → PATCH /leaves/:id/approve	React Component
14	authMiddleware	Verifies manager JWT	middleware/auth.middleware.js
15	roleMiddleware	allowRoles('admin','manager') — passes	middleware/role.middleware.js

Step	Actor / Layer	Action / Event	Tech Component
16	LeaveService	Updates status='approved', sets approvedBy=req.user.id	leaves.service.js
17	Response	Returns 200 to manager browser, leave status updated in UI	leaves.controller.js

10. Security Architecture & Best Practices

Security Concern	Implementation	Location
Password Storage	Bcrypt hashing with salt rounds (≥12)	AuthService + User schema (select: false)
Authentication	JWT with HS256 / RS256 signature, 24h expiry	auth.middleware.js + jwt.utils.js
Authorization	RBAC via allowRoles() on every protected route	role.middleware.js + all route files
Input Validation	express-validator on all POST/PATCH routes	validate.middleware.js (inline validators)
Transport Security	HTTPS enforced on all endpoints	Nginx + Vercel TLS
CORS	Whitelist only Vercel frontend origin	app.js (cors middleware config)
Token Storage	JWT stored in React AuthContext (memory only)	Frontend — never localStorage
Error Disclosure	Generic errors returned to client, full error only in logs	Global Error Handler + Winston
Rate Limiting	express-rate-limit on /auth/login (future)	Planned middleware
Pagination	All list endpoints paginated (?page=&limit=)	All GET list controllers

11. Backend Build Execution Order

Follow this exact sequence for a correct, dependency-respecting build:

Phase	Step	Task	Output
Foundation	1	MongoDB connection setup	config/db.js — mongoose.connect with retry logic
Foundation	2	User schema + model	modules/users/user.model.js with all fields + indexes
Auth	3	Auth service	auth.service.js — login(), hashPassword(), generateJWT()
Auth	4	Auth middleware	auth.middleware.js — verifyToken(), attach req.user
Auth	5	Role middleware	role.middleware.js — allowRoles() factory
Auth	6	Auth routes + controller	POST /auth/login fully working
Users	7	Create user endpoint	POST /users/create with role validation
Users	8	Profile endpoint	GET /profile returning req.user data
Tasks	9	Task schema + model	task.model.js with all fields + indexes
Tasks	10	Task service	CRUD + role-based filtering logic
Tasks	11	Task routes + controller	All /tasks/* endpoints
Leaves	12	Leave schema + model	leave.model.js with all fields + indexes
Leaves	13	Leave service	Apply + approve logic with role checks
Leaves	14	Leave routes + controller	All /leaves/* endpoints
Dashboard	15	Dashboard service	MongoDB aggregation pipelines per role
Dashboard	16	Dashboard route + controller	GET /dashboard/stats
Frontend	17	AuthContext + Axios interceptor	JWT storage + automatic header injection
Frontend	18	ProtectedRoute + RoleProtectedRoute	Route guards per role

Phase	Step	Task	Output
Frontend	19	Login page	POST /auth/login → role-based redirect
Frontend	20	Admin pages (Dashboard, Employees, Tasks, Leaves)	Full admin module
Frontend	21	Manager pages (Dashboard, Tasks, Leaves)	Full manager module
Frontend	22	Employee pages (Dashboard, Tasks, Leaves)	Full employee module

12. Standard Error Response Format

All error responses follow a consistent schema so the frontend can handle them uniformly:

```
// Standard error response shape
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR | UNAUTHORIZED | FORBIDDEN | NOT_FOUND |
SERVER_ERROR",
    "message": "Human-readable message safe for client display",
    "details": [ { "field": "email", "msg": "must be valid email" } ]  //
validation only
  }
}
```

HTTP Status	Error Code	When Triggered
400	VALIDATION_ERROR	express-validator fails on request body/params
401	UNAUTHORIZED	No token, expired token, or invalid token
403	FORBIDDEN	Valid token but insufficient role permissions
404	NOT_FOUND	Resource (user/task/leave) not found in DB
409	CONFLICT	Duplicate resource (e.g., email already exists)
500	SERVER_ERROR	Unexpected error — logged in Winston, generic message to client

13. Scalability & Infrastructure Notes

The architecture is designed for horizontal scalability from day one:

Concern	Current Approach	Scale Strategy
Multiple instances	Nginx load balancer distributes requests	Add Node.js instances on Render horizontally
Session state	Stateless JWT — no server-side sessions	Any instance can handle any request
Database	MongoDB Atlas — managed cluster	Scale reads with replica sets; writes via primary

Concern	Current Approach	Scale Strategy
Caching	Redis (optional) for dashboard aggregations	Cache invalidation on task/leave updates
Pagination	All list endpoints use ?page=&limit=	Prevents unbounded query results
Indexing	email, managerId, assignedTo, status indexed	Ensures O(log n) lookups on filtered queries

— END OF DOCUMENT —